

Manual de Instalação e Execução – Sistema PRODARTE

Este manual detalha o passo a passo para configurar, instalar e executar a plataforma **PRODARTE** do zero. O sistema é estruturado em um monorepo contendo:

- **Backend** (`apps/api/gestaoartesaos`): API REST construída com Java 17 e Spring Boot, usando banco relacional PostgreSQL e segurança JWT com chaves RSA (OAuth2).
 - **Frontend** (`apps/web/proarte-gestao-digital`): Pannel de gestão SPA construído com React, Vite, TypeScript e TanStack Start.
-



1. Pré-requisitos Gerais

Antes de iniciar a configuração do projeto, garanta que possui as seguintes ferramentas instaladas em sua máquina:

Desenvolvimento Local (Nativo)

- **Java Development Kit (JDK 17 ou superior)**: Necessário para compilar e rodar a API em Spring Boot.
- **Maven 3.8+** (opcional se utilizar o wrapper `./mvnw` incluso no repositório).
- **Node.js (Versão 20 LTS ou superior)** e gerenciador de pacotes **npm** (ou **Bun**) para executar o frontend.
- **PostgreSQL (Versão 15 ou superior)** rodando localmente (geralmente na porta `5432`).
- **OpenSSL**: Ferramenta de linha de comando para a geração das chaves RSA de autenticação.

Desenvolvimento via Contêineres

- **Docker** instalado e ativo.
 - **Docker Compose** instalado (geralmente empacotado junto com o Docker Desktop).
-



2. Estrutura de Pastas e Arquivos

O projeto é organizado no padrão monorepo para facilitar o compartilhamento de tipos e scripts. A estrutura principal é dividida da seguinte forma:

2.1. Backend (`apps/api/gestaoartesaos`)

- `Dockerfile` : Configuração multi-stage para compilação e execução da API dentro de um contêiner Docker.
- `pom.xml` : Gerenciador de dependências Maven do backend.
- `src/main/resources/` :
 - `application.properties` : Variáveis de configuração ativas da API.
 - `application.properties.example` : Modelo de configuração com placeholders.
 - `data.sql` : Script automático de carga inicial de dados simulados (artesãos, feiras e gestores).
 - `app.key` / `app.pub` : Chaves RSA locais geradas para assinatura de tokens.
- `src/main/java/com/prodarte/gestaoartesaos/` :
 - `GestaoartesaosApplication.java` : Classe principal de inicialização da API.
 - `configs/` : Definição de beans do Spring Security, CORS, codificador de senhas e filtros OAuth2.
 - `controllers/` : Endpoints REST expostos para o frontend (rotas de artesãos, feiras, login, etc.).
 - `dtos/` : Records/classes de transferência de dados de entrada e saída.
 - `enums/` : Definição de enums do domínio (ex: StatusCuradoria, Segmento).
 - `models/` : Entidades relacionais mapeadas com JPA para o banco PostgreSQL.
 - `repositories/` : Interfaces JPA que expõem operações de banco (Queries, Joins, etc.).
 - `services/` : Lógica central do sistema, cálculo de rodízio e conexões externas (WhatsApp via Twilio).
 - `specifications/` : Especificações para paginação, ordenação e filtros dinâmicos de artesãos.

2.2. Frontend (`apps/web/proarte-gestao-digital`)

- `package.json` : Dependências e scripts de execução (Vite, React, TanStack, Tailwind v4).
- `components.json` : Configuração para importação de componentes Shadcn UI.
- `vite.config.ts` : Configuração do bundler Vite contendo os plugins do TanStack Router.
- `src/` :
 - `components/` : Componentes visuais gerais e layouts do painel.
 - `app-layout.tsx` : Layout mestre com barra superior e controle de sessão do gestor.

- `app-sidebar.tsx` : Barra de navegação lateral para gerenciar feiras, triagens e mensagens.
 - `components/ui/` : Biblioteca Shadcn UI instalada localmente (botões, tabelas, inputs).
 - `lib/` :
 - `api-client.ts` : Utilitários do Axios / chamadas de integração HTTP com a API.
 - `store.ts` : Gerenciamento de estado global da aplicação usando Zustand (Sessão, Cache, etc.).
 - `routes/` : Roteamento baseado em arquivos com o TanStack Router:
 - `__root.tsx` : Layout raiz de controle geral de HTML/head.
 - `index.tsx` : Tela de login/autenticação dos gestores.
 - `dashboard.tsx` : Painel principal de Triagem e Curadoria de artesãos.
 - `feiras.tsx` : Painel de visualização, criação e alocação de feiras e rodízio.
 - `mensageria.tsx` : Painel de controle e disparo de mensagens em lote ou individuais via WhatsApp.
 - `privacy.tsx` : Página pública contendo a Política de Privacidade e conformidade com a LGPD.
 - `styles.css` : Importações globais do Tailwind CSS v4 e animações de transição.
-



3. Geração das Chaves RSA para OAuth2 (JWT)

A API do PRODARTE utiliza criptografia assimétrica (chaves RSA) para assinar e validar os tokens JWT do sistema. Para que a autenticação funcione, é obrigatório gerar um par de chaves (pública e privada).

Passo a passo para gerar as chaves via OpenSSL:

Abra o terminal e execute os seguintes comandos:

```
# 1. Gerar uma chave privada RSA de 2048 bits
openssl genrsa -out keypair.pem 2048
```

```
# 2. Converter a chave privada para o formato PKCS#8 (necessário para lei
openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt -in keypair.pem -c
```

```
# 3. Extrair a chave pública correspondente no formato PEM
openssl rsa -in keypair.pem -pubout -outform PEM -out app.pub
```

```
# 4. Excluir o arquivo temporário de chaves
# No Linux / macOS:
rm keypair.pem
```

```
# No Windows (PowerShell):  
Remove-Item keypair.pem
```

Onde colocar os arquivos gerados?

Mova os arquivos `app.key` e `app.pub` para a pasta de recursos do backend:

 `apps/api/gestaoartesaos/src/main/resources/`

4. Configurações de Ambiente (Variáveis)

4.1. Backend (`application.properties`)

No backend, as configurações são feitas no arquivo

`apps/api/gestaoartesaos/src/main/resources/application.properties` . Crie um ou edite a partir do arquivo `.example` :

Propriedade	Descrição	Valor de Exemplo / Padrão
<code>jwt.public.key</code>	Caminho para a chave pública	<code>classpath:app.pub</code>
<code>jwt.private.key</code>	Caminho para a chave privada	<code>classpath:app.key</code>
<code>spring.datasource.url</code>	URL de conexão com o PostgreSQL	<code>jdbc:postgresql://localhost:5432/gestaoartesaos</code>
<code>spring.datasource.username</code>	Usuário do banco de dados	<code>postgres</code>
<code>spring.datasource.password</code>	Senha do banco de dados	<code>sua-senha-aqui</code>
<code>spring.sql.init.mode</code>	Modo de carga inicial do banco	<code>always</code> (popula tabelas a partir do <code>data.sql</code> no início)
<code>twilio.account-sid</code>	SID da conta Twilio para WhatsApp	Credencial da API Twilio
<code>twilio.auth-token</code>	Token de autenticação da Twilio	Credencial da API Twilio
<code>twilio.whatsapp-number</code>	Número oficial de disparo (WhatsApp)	<code>whatsapp:[seu-numero]</code>

4.2. Frontend (`.env`)

No frontend, configure o arquivo `.env` dentro da pasta `apps/web/proarte-gestao-digital/` :

Variável	Descrição	Valor de Exemplo / Padrão
VITE_API_URL	URL de comunicação com a API Backend	http://localhost:8080

5. Como Executar o Backend (Duas Formas)

Antes de iniciar, certifique-se de que a sua porta `8080` está livre.

Forma A: Execução Local (Nativo)

Passo a Passo:

1. **Criar o Banco de Dados:** Apenas crie um banco de dados vazio chamado `gestaoartesaos` no PostgreSQL:

```
CREATE DATABASE gestaoartesaos;
```

2. **Configurar as Propriedades:** Configure o `application.properties` com as credenciais do seu PostgreSQL.
3. **Chaves OAuth:** Garanta que as chaves `app.key` e `app.pub` geradas no passo 3 estejam em `src/main/resources/`.

4. **Rodar a aplicação:**

Abra o terminal em `apps/api/gestaoartesaos` e execute:

```
# No Linux / macOS:
./mvnw spring-boot:run

# No Windows (PowerShell/CMD):
.\mvnw.cmd spring-boot:run
```

A API estará ativa em `http://localhost:8080`.

Forma B: Execução via Docker (Recomendado)

Opção B.1: Usando Docker Compose (Fácil e Automatizado)

Crie um arquivo chamado `docker-compose.yml` na raiz do projeto com a seguinte configuração:

version: '3.8'

services:

database:

image: postgres:15-alpine

container_name: prodarte-postgres

environment:

POSTGRES_DB: gestaoartesaos

POSTGRES_USER: postgres

POSTGRES_PASSWORD: password123

ports:

- "5432:5432"

volumes:

- pgdata:/var/lib/postgresql/data

networks:

- prodarte-net

api:

build:

context: ./apps/api/gestaoartesaos

dockerfile: Dockerfile

container_name: prodarte-api

ports:

- "8080:8080"

environment:

SPRING_DATASOURCE_URL: jdbc:postgresql://database:5432/gestaoartesaos

SPRING_DATASOURCE_USERNAME: postgres

SPRING_DATASOURCE_PASSWORD: password123

SPRING_SQL_INIT_MODE: always

JWT_PUBLIC_KEY: classpath:app.pub

JWT_PRIVATE_KEY: classpath:app.key

depends_on:

- database

networks:

- prodarte-net

volumes:

pgdata:

networks:

prodarte-net:

driver: bridge

Para rodar todo o ambiente backend com o Compose:

```
docker-compose up --build -d
```

Opção B.2: Comandos Docker Isolados

Se preferir rodar os contêineres manualmente sem compose:

1. Subir contêiner do PostgreSQL:

```
docker run --name prodarte-db -d -p 5432:5432 -e POSTGRES_DB=gestaoa
```

2. Buildar a Imagem da API:

Navegue até `apps/api/gestaoartesaos` e execute o comando:

```
docker build -t prodarte-api .
```

3. Rodar a API (conectando ao contêiner de banco):

```
docker run --name prodarte-api -d -p 8080:8080 --link prodarte-db:dat
```



6. Como Executar o Frontend (Vite / React)

Passo a Passo:

1. **Configurar as variáveis:** Crie o arquivo `.env` na pasta `apps/web/proarte-gestao-digital/` e aponte para o backend:

```
VITE_API_URL=http://localhost:8080
```

2. **Instalar dependências:** No terminal, navegue até `apps/web/proarte-gestao-digital` e instale os pacotes:

```
npm install
```

```
# ou caso use o Bun:
```

```
bun install
```

3. **Executar em modo desenvolvimento:**

```
npm run dev
# ou caso use o Bun:
bun dev
```

O painel estará disponível no navegador em `http://localhost:5173`.

? 7. Dúvidas Frequentes e Resolução de Problemas

7.1. Como fazer login na primeira execução?

O banco é populado automaticamente via script `data.sql`. O usuário gestor inicial pré-configurado é:

- **E-mail:** `gestor@prodarte.com`
- **Senha:** `senha123` (esta senha é a correspondente em texto claro para o hash criptografado `$2a$10$TMxK9prKz/wqkGhByYdn0u9dxAC638g4dZlNApPNHaZDHpfRKiIDy` inserido no banco).

Você também pode registrar outros usuários gestores fazendo uma requisição `POST` pública para o endpoint `/usuario`.

7.2. Ocorreu erro de CORS ao conectar o frontend com a API

A API do Spring Boot tem proteção e liberação de CORS integrada em `SecurityConfig.java`. Por padrão, ela aceita requisições vindas de:

- `http://localhost:*` (Qualquer porta local para desenvolvimento)
- `http://127.0.0.1:*`
- `https://prodarte-test.netlify.app` (Ambiente de homologação em nuvem)

Se você hospedar o frontend em outra URL, certifique-se de adicioná-la à lista

`setAllowedOriginPatterns` no método `corsConfigurationSource()` da classe `SecurityConfig.java`.

7.3. Erro "Invalid key" ou "Private key cannot be read" no início do Backend

Isso ocorre quando a chave privada gerada não está no formato PKCS#8 aceito pelo Java Security.

Certifique-se de que utilizou a instrução correta do OpenSSL convertendo com o comando `openssl`

`pkcs8 -topk8 -inform PEM -outform PEM -nocrypt -in keypair.pem -out app.key`. O cabeçalho do arquivo final `app.key` deve iniciar exatamente com:
`-----BEGIN PRIVATE KEY-----` (e não `-----BEGIN RSA PRIVATE KEY-----`).

7.4. O banco de dados não popula com os dados simulados

A inicialização do banco depende da diretiva `spring.sql.init.mode=always`. Caso as tabelas já tenham sido criadas e você precise reiniciar a carga simulada do zero, configure `spring.jpa.hibernate.ddl-auto=create-drop` temporariamente para que o Hibernate recrie o esquema e o script `data.sql` seja executado novamente limpo.

7.5. Erros de conexão com o banco de dados ("Connection refused")

- Verifique se o serviço do PostgreSQL local está ativo rodando o comando `pg_isready` ou abrindo o pgAdmin/DBeaver.
- Se estiver usando Docker, garanta que o contêiner do PostgreSQL (`prodarte-postgres`) foi iniciado corretamente antes da API. O `depends_on` no Compose auxilia nesse processo, mas às vezes o banco de dados demora alguns segundos adicionais para aceitar conexões.
- Verifique se as credenciais (username e password) e o nome do banco no `application.properties` correspondem exatamente às do banco local criado.

7.6. Portas em conflito (8080 ou 5173 ocupadas)

- **Backend (8080):** Caso outra aplicação utilize a porta `8080`, você pode definir uma nova porta para a API adicionando a propriedade `server.port=8081` no arquivo `application.properties`. Lembre-se de ajustar também a variável `VITE_API_URL` no frontend para apontar para a nova porta (`http://localhost:8081`).
- **Frontend (5173):** O Vite automaticamente tentará a próxima porta disponível (ex: `5174`) se a `5173` estiver em uso. No entanto, se precisar fixar a porta, adicione `--port 3000` na linha de execução do script dev no `package.json`.

7.7. Expiração de Token JWT / Sessão inválida (HTTP 401 Unauthorized)

Os tokens JWT gerados possuem expiração padrão configurada para 3600 segundos (1 hora). Se você receber erros HTTP 401 ao fazer chamadas a endpoints protegidos, a sessão do gestor expirou. O frontend detecta essa rejeição e solicita que o usuário realize um novo login na tela inicial.